

INFORMAÇÕES

MÓDULO	AULA	INSTRUTOR
09 Introdução ao React para desenvolvedores .NET	04 - Componentes Props e Composição de Interface com React e TypeScript	Pedro Henrique Amadio

CONTEÚDO

1. Um pulinho na aula anterior...

Na aula passada, tivemos o primeiro contato com React + TypeScript, entendendo o que é **React**, o que é **JSX/TSX**, o conceito de **componente**, além do uso inicial de **useState** e uma introdução ao **useEffect**. No nosso hands-on, criamos nosso primeiro projeto com **Vite** e **React**, com construção de uma interface simples de lista de tarefas.

Agora, nesta aula, o foco será dar um passo muito importante dentro do **React**: **aprender a separar a interface em componentes reutilizáveis**.

Essa é uma das ideias mais fortes do **React** e uma das razões pelas quais ele é tão utilizado no front-end moderno: construir telas por partes menores, reaproveitáveis e organizadas. Inclusive, própria **Aula 03** já introduz o **React** como uma biblioteca baseada em components e a **Aula 04** aprofunda exatamente esse ponto na prática.

2. O que veremos hoje?

Nesta aula, vamos trabalhar três pilares:

1. Props

Entender o que são, para que servem, como tipar e como receber funções via **props**.

2. Botão reutilizável

Criar um componente de botão com propriedades como:

- label
- disabled
- isLoading
- isLoadingText
- onClick

3. Modal com CSS puro

Criar um componente modal reutilizável usando:

- **props**
- **useState**

- funções de abrir e fechar
- composição de interface
- estilização com CSS puro

3. O que são componentes?

Componentes são partes reutilizáveis da interface.

Em vez de escrever toda a tela dentro de um único arquivo grande, podemos quebrar a interface em pedaços menores, cada um com sua responsabilidade.

Exemplos de componentes:

- botão;
- modal;
- cabeçalho;
- formulário;
- card;
- item de lista.

Isso melhora:

- organização;
- leitura;
- manutenção;
- reaproveitamento;
- crescimento do projeto.

Na **Aula 03**, o **React** já foi apresentado como uma biblioteca baseada em componentes, incentivando a divisão da interface em pequenas unidades visuais e lógicas.

Exemplo simples:

```
function Titulo() {  
  return <h1>Minha aplicação</h1>;  
}
```

Aqui, Titulo é um componente.

4. O que são props?

Props é o nome dado aos dados que um componente recebe.

Podemos pensar assim:

- o componente é uma peça reutilizável;
- as **props** são os valores de configuração dessa peça.

Por exemplo, um botão pode mudar de texto, cor, comportamento ou estado dependendo das **props** que recebe.

Exemplo conceitual:

```
function BoasVindas(props: { nome: string }) {  
  return <h1>Olá, {props.nome}!</h1>;  
}
```

Uso:

```
<BoasVindas nome="Pedro" />
```

Nesse caso:

- o componente é BoasVindas;
- a **prop** é nome;
- o valor enviado foi "Pedro".

5. Como definir props em React com TypeScript?

Em **React** com **TypeScript**, é uma boa prática definir o tipo das **props**. Podemos fazer isso com **type** ou **interface**.

Exemplo com type:

```
type BoasVindasProps = {  
  nome: string;  
};  
  
function BoasVindas({ nome }: BoasVindasProps) {  
  return <h1>Olá, {nome}!</h1>;  
}
```

Exemplo com interface:

```
interface BoasVindasProps {  
  nome: string;  
}  
  
function BoasVindas({ nome }: BoasVindasProps) {  
  return <h1>Olá, {nome}!</h1>;  
}
```

As duas abordagens funcionam bem.

6. Props comuns: string, boolean e children

As **props** podem receber diferentes tipos de dados.

String

```
type CardProps = {  
  titulo: string;  
};  
  
function Card({ titulo }: CardProps) {  
  return <h2>{titulo}</h2>;  
}
```

Boolean

```
type AlertaProps = {
  ativo: boolean;
};

function Alerta({ ativo }: AlertaProps) {
  return <p>{ativo ? "Ativo" : "Inativo"}</p>;
}
```

Children

children representa o conteúdo interno passado entre a abertura e fechamento da tag do componente.

```
type CaixaProps = {
  children: React.ReactNode;
};

function Caixa({ children }: CaixaProps) {
  return <div>{children}</div>;
}
```

Uso:

```
<Caixa>
  <p>Conteúdo dentro da caixa</p>
</Caixa>
```

7. Props com funções

Um ponto muito importante no **React** é que **props** também podem receber **funções**. Isso permite que um componente filho avise algo para o componente pai.

Esse padrão é muito usado em:

- botões;
- formulários;
- modais;
- listas;
- callbacks em geral.

Exemplo:

```
type BotaoProps = {
  onClick: () => void;
};

function Botao({ onClick }: BotaoProps) {
  return <button onClick={onClick}>Clique aqui</button>;
}
```

Uso:

```
function App() {
  const mostrarMensagem = () => {
    alert("Botão clicado!");
  }
}
```

```
    return <Botao onClick={mostrarMensagem} />;  
  }  
}
```

Aqui temos um ponto essencial:

- App criou a função;
- Botao recebeu essa função via **prop**;
- ao clicar no botão, o componente executa a função recebida.

Isso é um **callback**.

8. O que são callbacks?

Callback é uma função passada como argumento ou **prop** para ser executada depois, em resposta a algum evento.

No **React**, isso acontece o tempo todo.

Exemplos comuns:

- `onClick`
- `onChange`
- `onClose`
- `onSubmit`

Exemplo:

```
type ModalProps = {  
  onClose: () => void;  
};  
  
function Modal({ onClose }: ModalProps) {  
  return <button onClick={onClose}>Fechar modal</button>;  
}
```

Nesse caso, o componente Modal não decide sozinho o que acontece ao fechar.

Ele apenas executa a função recebida.

Isso deixa o componente mais reutilizável e mais flexível.

9. Hands-on

Agora que vimos como componentes são criados e como definimos **props** para os componentes, está na hora de botar a mão na massa!

Estrutura sugerida de pastas

```
src/  
  components/  
    Button.tsx  
    Modal.tsx  
  App.tsx  
  main.tsx  
  index.css
```

1. Criando nosso primeiro componente de botão reutilizável

Criaremos um botão que aceite props como (**Button.tsx**):

- `label`
- `disabled`
- `isLoading`

- isLoadingText
- onClick

Estrutura do type

```
type ButtonProps = {  
  label: string;  
  disabled?: boolean;  
  isLoading?: boolean;  
  isLoadingText?: string;  
  onClick?: () => void;  
};
```

Relembrando:

Observe que algumas props têm ?.
Isso significa que elas são **opcionais**.

Estrutura final do **Button.tsx**

```
type ButtonProps = {  
  label: string;  
  disabled?: boolean;  
  isLoading?: boolean;  
  isLoadingText?: string;  
  onClick?: () => void;  
};  
  
function Button({  
  label,  
  disabled = false,  
  isLoading = false,  
  isLoadingText = "Carregando...",  
  onClick  
}: ButtonProps) {  
  return (  
    <button  
      className="custom-button"  
      onClick={onClick}  
      disabled={disabled || isLoading}  
    >  
      {isLoading ? isLoadingText : label}  
    </button>  
  );  
}  
  
export default Button;
```

CSS do Botão

```
.custom-button {  
  padding: 10px 16px;  
  border: none;  
  border-radius: 8px;  
  background-color: #2563eb;  
  color: white;  
  font-size: 14px;
```

```

    cursor: pointer;
    transition: opacity 0.2s ease;
  }

  .custom-button:hover {
    opacity: 0.9;
  }

  .custom-button:disabled {
    opacity: 0.6;
    cursor: not-allowed;
  }

```

2. Criando nosso primeiro Modal

Criaremos um modal que terá parâmetros de controle de exibição e funções de **callback**.

Estrutura do **Modal.tsx**

```

type ModalProps = {
  isOpen: boolean;
  title: string;
  children: React.ReactNode;
  onClose: () => void;
};

function Modal({ isOpen, title, children, onClose }: ModalProps) {
  if (!isOpen) {
    return null;
  }

  return (
    <div className="modal-overlay">
      <div className="modal-box">
        <div className="modal-header">
          <h2>{title}</h2>
          <button className="modal-close-button" onClick={onClose}>
            X
          </button>
        </div>

        <div className="modal-content">
          {children}
        </div>
      </div>
    </div>
  );
}

export default Modal;

```

Explicando a lógica do modal

```
if (!isOpen) return null;
```

Essa linha é muito importante.

Se o modal não estiver aberto, ele não renderiza nada.

Ou seja:

- true: modal aparece;
- false: modal não aparece.

children

Permite que o componente seja reaproveitado com conteúdos diferentes.

Exemplo:

```
<Modal ...>  
  <p>Texto interno</p>  
</Modal>
```

onClose

É a função que será executada ao clicar no botão de fechar.

CSS do modal

```
.modal-overlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background-color: rgba(0, 0, 0, 0.5);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}  
  
.modal-box {  
  background-color: white;  
  width: 400px;  
  max-width: 90%;  
  border-radius: 12px;  
  padding: 20px;  
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.25);  
}  
  
.modal-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  margin-bottom: 16px;  
}  
  
.modal-close-button {  
  border: none;  
  background: transparent;  
  font-size: 18px;  
  cursor: pointer;  
}  
  
.modal-content {  
  font-size: 14px;
```



```
    line-height: 1.5;
  }
```

3. Usando os componentes de Botão com o Modal

Agora vamos controlar a abertura e o fechamento do modal pelo componente principal.

```
import { useState } from "react";
import Button from "../components/Button";
import Modal from "../components/Modal";
import "../index.css"

function App() {
  const [isModalOpen, setIsModalOpen] = useState<boolean>(false);

  function abrirModal(): void {
    setIsModalOpen(true);
  }

  function fecharModal(): void {
    setIsModalOpen(false);
  }

  return (
    <div>
      <h1>Componentes com React + TypeScript</h1>

      <Button label="Abrir modal" onClick={abrirModal} />

      <Modal
        isOpen={isModalOpen}
        title="Meu primeiro modal"
        onClose={fecharModal}
      >
        <p>Este é um modal criado com React, props, useState e CSS puro.</p>
      </Modal>
    </div>
  );
}

export default App;
```

4. Versão alternativa

App.tsx

```
import { useState } from "react";
import Button from "../components/Button/Button";
import Modal from "../components/Modal/Modal";
import "../index.css"

function App() {
  const [isModalOpen, setIsModalOpen] = useState<boolean>(false);
  const [isSaving, setIsSaving] = useState<boolean>(false);
```

```

function abrirModal(): void {
  setIsModalOpen(true);
}

function fecharModal(): void {
  setIsModalOpen(false);
}

function simularSalvar(): void {
  setIsSaving(true);

  setTimeout(() => {
    alert("Salvo com sucesso!");
    setIsSaving(false);
  }, 2000);
}

return (
  <div style={{ padding: "24px" }}>
    <h1>Aula 04 - Componentes e Props</h1>
    <p>Hoje vamos trabalhar com componentes reutilizáveis.</p>

    <div style={{ display: "flex", gap: "12px", marginTop: "16px" }}>
      <Button label="Abrir modal" onClick={abrirModal} />
      <Button
        label="Salvar"
        onClick={simularSalvar}
        isLoading={isSaving}
        isLoadingText="Salvando..."
      />
      <Button label="Desabilitado" disabled />
    </div>

    <Modal
      isOpen={isModalOpen}
      title="Exemplo de modal"
      onClose={fecharModal}
    >
      <p>Este modal foi criado com props, useState e CSS puro.</p>
      <p>Também usamos uma função de callback para fechá-lo.</p>
    </Modal>
  </div>
);
}

export default App;

```

10. Exercícios de fixação

1. O que são props em React?

- () São arquivos CSS obrigatórios
- () São valores que um componente recebe

- ☐ () São comandos do Vite
- ☐ () São apenas funções internas do React

2. Qual prop normalmente usamos para receber uma função de clique?

- ☐ () label
- ☐ () title
- ☐ () onClick
- ☐ () children

3. O que significa esta tipagem?

```
onClose: () => void;
```

- ☐ () Uma string obrigatória
- ☐ () Uma função sem retorno
- ☐ () Um array de funções
- ☐ () Um objeto opcional

4. O que faz a linha abaixo no modal?

```
if (!isOpen) {  
  return null;  
}
```

- ☐ () Fecha o navegador
- ☐ () Renderiza o modal duas vezes
- ☐ () Impede a renderização quando o modal estiver fechado
- ☐ () Reinicia o estado da aplicação

5. Qual a vantagem de criar um botão como componente reutilizável?

- ☐ () O código fica maior
- ☐ () O botão só funciona com modal
- ☐ () Podemos reutilizar lógica e aparência em vários lugares
- ☐ () O React deixa de usar props

11. Desafio para casa

Criar uma pequena tela contendo:

- um botão “Abrir detalhes”;
- um modal com título e texto;
- um botão “Enviar” com estado de loading;
- um botão desabilitado;
- estilização com CSS puro.

Regras

- tipar todas as props;
- usar `useState` no componente principal;
- usar função de callback para fechar o modal;
- não deixar o botão de loading clicável enquanto estiver carregando.